

PROGRAMMAZIONE II

ARRAY E TIPI ENUMERATIVI

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.A. 2025/2026

ARRAY



ARRAY IN JAVA

Un **array** è una sequenza ordinata di variabili dello **stesso tipo** a cui si accede tramite un indice (un numero naturale)

- ▶ Può contenere sia tipi primitivi che riferimenti a oggetti
- ▶ Non può contenere valori oggetto
- ▶ Gli indici degli array in Java partono da 0

// come dovrebbe sempre essere

La dimensione dell'array può essere definita a **run-time**, durante la creazione dell'oggetto

- ▶ Ma non può cambiare successivamente

ARRAY IN JAVA (CONT.)

Gli array in Java sono oggetti speciali (allocati nell'heap)

- La dichiarazione di un array alloca un **riferimento**

`int arr[];` ----- `int[] arr;`

equivalente

Non è necessario specificare la dimensione, poiché stiamo dichiarando un riferimento

Riferimento a un array di `int`
(inizializzato a `null`)

CREAZIONE DI ARRAY

Con l'operatore `new`

```
float[] arr = new float[10]; // alloca un array di 10 elementi      1
int size = 7;                // gli elementi assumono il valore di default 2
float[] arr = new float[size]; // dimensione dell'array fornita a runtime 3
```

Con inizializzazione statica

```
int[] primes = new int[]{2, 3, 5}; // dimensione dedotta automaticamente 1
int[] primes = {2, 3, 5};           // notazione più breve 2
Car[] cars = {new Car("red"), myCar}; // ammessi valori o riferimenti 3
```

ACCESSO AGLI ARRAY

Gli elementi dell'array sono letti con le parentesi []

► I limiti dell'array sono controllati a runtime

```
int[] primes = new int[]{2, 3, 5};           1
System.out.println(primes[1]); // l'output sarà '3'  2
System.out.println(primes[6]); // la JVM lancerà un'eccezione  3
```

Gli elementi dell'array sono scritti con le parentesi []

```
Car[] cars = new Car[3];                      1
cars[1] = new Car("red"); // cars punta a [null , Car@25c7f37d , null]  2
```

OPERAZIONI SUGLI ARRAY

Ogni array è dotato di uno (pseudo) campo `length`

```
int[] primes = new int[]{2, 3, 5, 7};  
System.out.println(primes.length); // l'output sarà '4'
```

1

2

Se l'array non è inizializzato o è `null`, `length` non è accessibile

```
int[] primes = null;  
System.out.println(primes.length); // la JVM lancerà un'eccezione
```

1

2

OPERAZIONI SUGLI ARRAY (CONT.)

Come per qualsiasi oggetto, `==` si applica al riferimento e non agli elementi dell'array

- Il metodo `equals()` non funziona per gli array

```
int[] arr1 = {1,2,3}, arr2 = {1,2,3};           1
System.out.println(arr1 == arr2);                // l'output è 'false '      2
System.out.println(arr1.equals(arr2));           // l'output è 'false '      3
System.out.println(Arrays.equals(arr1, arr2));    // l'output è 'true '       4
```

Usare il metodo `Arrays.equals()` del pacchetto `java.util`

- È necessario importare la classe `java.util.Arrays`

OPERAZIONI SUGLI ARRAY (CONT.)

Lo stesso vale per il metodo `toString()` degli array

- ▶ Stampa il riferimento e non il contenuto dell'array

```
int[] arr = {1,2,3};                                1
System.out.println(arr.toString());                  2
System.out.println(Arrays.toString(arr));             3
```

// l'output è '[I@6ce253f1 '

// l'output è '[1, 2, 3]'

Usare il metodo `Arrays.toString()` del pacchetto `java.util`

- ▶ È necessario importare la classe `java.util.Arrays`

CICLO ENHANCED FOR

Nuovo costrutto di ciclo disponibile a partire da Java 5

`for`(*Type var : IterExpr*)

dove *IterExpr* può essere

- Un array o una classe iterabile

// vedremo gli iterabili più avanti

```
for (String arg : args)           // stampa gli argomenti del programma    1
    System.out.println(arg);      2
                                   3
for (int i = 0; i < args.length; i++) {  4
    String arg = args[i];          5
    System.out.println(arg);      // i due cicli sono equivalenti          6
}                                   7
```

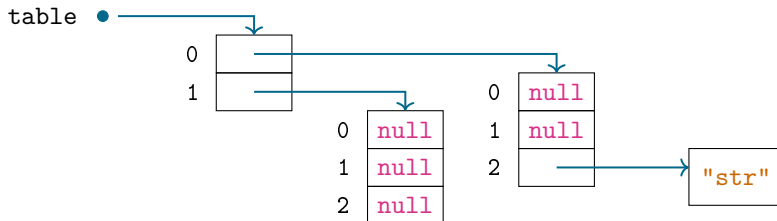
ARRAY MULTIDIMENSIONALI

Implementati come array di array

```
String[][] table = new String[2][3];  
table[0][2] = new String("str");
```

1

2



ARRAY MULTIDIMENSIONALI (CONT.)

Le righe non sono memorizzate in posizioni adiacenti in memoria

- Possono essere scambiate facilmente
- Possono avere lunghezze diverse

```
int[][] matrix = { {1,2,3}, {4,5,6}, {7,8,9} };           1
int[] temp = matrix[0];                                   2
matrix[0] = matrix[matrix.length-1];                     3
matrix[matrix.length-1] = temp; // prima riga scambiata con l'ultima 4
                                                            5
int[][] pyramid new int[3][]; // pyramid è:              6
pyramid[0] = new int[1]; // [null]                        7
pyramid[1] = new int[2]; // [null, null]                  8
pyramid[2] = new int[3]; // [null, null, null]            9
```

ANCORA PROBLEMI DI CONFRONTO E STRINGHE

Gli array multidimensionali **non dovrebbero** essere confrontati

- ▶ Usando `==`, `equals()` e anche `Arrays.equals()`
- ▶ Lo stesso vale per i metodi `toString()` e `Arrays.toString()`

Il programmatore deve iterare sulle righe

```
int [][] matrix = { {1,2,3}, {4,5,6}, {7,8,9} };
System.out.println(Arrays.toString(matrix));
// l'output è '[ [ I@53d8d10a , [ I@e9e54c2 , [ I@65ab7765] ]'

for (int[] row : matrix) {
    System.out.println(Arrays.toString(row));
}
```

TIPI ENUMERATIVI



ENUM

I **tipi enumerativi** consentono alle variabili di variare su un insieme prefissato di costanti

- ▶ Introdotti in Java 5 con la parola chiave `enum`
- ▶ Le variabili possono assumere solo uno dei valori enumerati

```
public enum Direction {  
    NORTH, SOUTH, EAST, WEST  
}  
  
Direction move = Direction.WEST;
```

1
2
3
4
5

ENUM (CONT.)

I tipi enumerativi possono essere dichiarati

- ▶ All'interno di una classe o in un file separato (come qualsiasi altra classe)
- ▶ Ma non all'interno di un metodo

I tipi enumerativi non sono stringhe o numeri

- ▶ Classi speciali il cui costruttore non può essere invocato direttamente
- ▶ Ma possiedono una rappresentazione testuale e una numerica

```
public enum Direction { NORTH, SOUTH, EAST, WEST }           1
Direction move = Direction.WEST;                               2
                                                                3
System.out.println(move.name()); // l'output è 'WEST'         4
System.out.println(move.ordinal()); // l'output è '3'          5
```


CONFRONTO DI ENUM

I tipi enumerativi **possono** essere confrontati in sicurezza con ==

- ▶ Gli enum garantiscono che esista una sola istanza delle costanti nella JVM
- ▶ In realtà, è più sicuro di equals()

```
public enum Direction { NORTH, SOUTH, EAST, WEST }           1
Direction move1 = Direction.WEST, move2 = null;               2
                                                                3
System.out.println(move1 == Direction.WEST); // l'output è 'true ' 4
System.out.println(move2 == Direction.WEST); // l'output è 'false ' 5
System.out.println(move2.equals(Direction.WEST)); // eccezione della JVM 6
```

USO DEGLI ENUM

Gli enum possono essere usati nelle istruzioni `switch`

```
public String moveDirectionAsString(Direction move) {
    switch (move) {
        case NORTH: return "↑";
        case SOUTH: return "↓";
        case EAST: return "→";
        case WEST: return "←";
    }

    return null;
}
```

USO DEGLI ENUM (CONT.)

Gli enum possono essere usati nei cicli enhanced for

```
public void printDirections() {  
    for (Direction d : Direction.values()) {  
        System.out.println(d.name());  
    }  
}
```

1
2
3
4
5
6
7



DOMANDE E RISPOSTE

